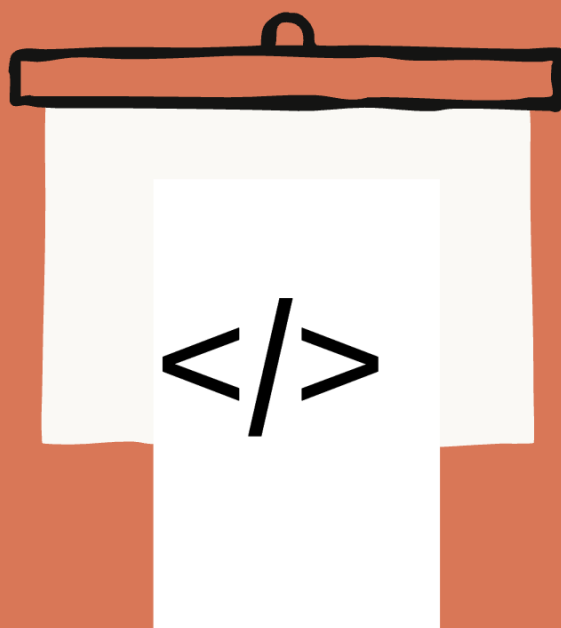


Anthropic 团队如何使用

— — — —



Anthropic 的内部团队正借助 Claude Code 改造工作流程，让开发人员和非技术人员能够处理复杂项目、自动化任务，并填补此前限制其工作效率的技能短板。

通过与我们自己的 Claude Code 核心用户进行访谈，我们收集了有关不同部门如何利用 Claude Code、对他们工作产生的影响，以及为其他考虑采用该工具的组织提供的建议。



Claude Code 数据基础设施	3
Claude Code 产品开发	5
Claude Code 安全工程	7
Claude Code 推理	9
Claude Code 数据科学与可视化	11
Claude Code API	13
Claude Code 增长营销	15
Claude Code 产品设计	17
Claude Code 强化学习工程	19
Claude Code 法务	21

用于数据基础设施的

数据基础设施团队为公司各团队整理所有业务数据。他们借助 Claude Code 自动化日常数据工程任务、排查复杂的基础设施问题，并为技术和非技术团队成员创建标准化工作流程文档，让他们能够独立访问和处理数据。

Claude Code 主要应用场景

带截图的 Kubernetes 调试

当 Kubernetes 集群出现故障且无法调度新 Pod 时，团队使用 Claude Code 诊断问题。他们将仪表板的截图输入 Claude Code，该工具会逐菜单引导他们完成 Google Cloud 的用户界面菜单，直到发现 Pod IP 地址耗尽的警告。随后 Claude Code 提供了创建新 IP 池并将其添加到集群的具体命令，无需让网络专家参与。

面向财务团队的纯文本工作流

该团队向财务团队成员展示了如何编写描述其数据工作流的纯文本文件，然后将这些文件加载到 Claude Code 中以实现完全自动化执行。没有编程经验的员工只需描述“查询此仪表板、获取信息、执行这些查询、生成 Excel 输出”之类的步骤，Claude Code 就会执行整个工作流，包括请求日期等必要输入。

新员工的代码库导航

当新的数据科学家加入团队时，他们会被指导使用 Claude Code 来浏览庞大的代码库。Claude Code 会读取他们的 Claude.md 文档文件，针对特定任务识别相关文件，解释数据管道的依赖关系，并帮助新成员了解哪些上游数据源为各类仪表盘提供支持。这一工具取代了传统的数据目录和可发现性工具。

会话结束时的文档更新

团队要求 Claude Code 在每个任务结束时总结已完成的工作环节并提出改进建议。这形成了一个持续改进的循环，Claude Code 根据实际使用情况优化 Claude.md 文档和工作流程说明，让后续的迭代更具成效。

跨多个实例的并行任务管理

在处理长期运行的数据任务时，他们会在不同代码仓库中为不同项目打开多个 Claude Code 实例。每个实例都保留完整上下文，因此当他们在数小时或数天后切回时，Claude Code 能准确记住他们正在做的工作以及中断的位置，实现真正的并行工作流管理且不会丢失上下文。

用于数据基础设施的

团队影响力

无需专业背景知识即可解决基础设施问题

借助 Claude Code 诊断问题并提供精准解决方案，解决了原本需要引入系统或网络团队成员的 Kubernetes 集群问题。

加速入职流程

新的数据分析师和团队成员无需大量指导，就能快速理解复杂系统并做出有价值的贡献。

简化了复杂工作流程

能够处理大得多的数据量，并识别出人工审核时人类无法完成的异常情况（比如监控 200 个仪表盘）。

无缝跨团队协作

没有编程经验的财务团队现在可以独立执行复杂的数据 workflows。

数据基础设施团队的核心技巧

撰写详细的 Claude.md 文件

你在 Claude.md 文件中对工作流程、工具和预期的记录越完善，Claude Code 的表现就越好。这让 Claude Code 能够出色地完成常规任务，比如在你已有现有模式的情况下搭建新的数据管道。

针对敏感数据 使用 MCP 服务器而非命令行界面

他们建议使用 MCP 服务器而非 BigQuery 命令行界面，以更好地控制 Claude Code 的访问权限，尤其是在处理需要日志记录或存在潜在隐私问题的敏感数据时。

公开团队使用公开

团队举办了分享会，成员们在会上向彼此展示了自己的 Claude Code 工作流程。这有助于推广最佳实践，也展示了使用该工具的多种不同方式，而这些方式他们可能无法靠自己发现。

用于产品开发的 Claude Code

Claude Code 团队使用自家产品来为 Claude Code 开发更新，以此拓展该产品的企业级功能和智能体循环功能。

Claude Code 主要应用场景

自动接受模式下的快速原型设计

工程师们借助“自动接受模式”（Shift+Tab）启用 Claude Code 来进行快速原型设计，并搭建自主循环流程，让 Claude 编写代码、运行测试并持续迭代。他们会向 Claude 提出自己不熟悉的抽象问题，交由其自主完成工作，随后在接手进行最终优化前，先审查完成度 80% 的解决方案。团队强调要从干净的 Git 状态起步，并定期提交节点版本，这样一旦 Claude 的开发方向出现偏差，就能轻松回退任何错误的修改。

核心功能的同步验证

对于涉及应用程序业务逻辑的更关键功能，团队与 Claude Code 同步协作，提供包含具体实现说明的详细提示词。他们实时监控整个过程，以确保代码质量、符合风格指南且架构合理，同时让 Claude 处理重复性的编码工作。

开发 Vim 模式

他们最成功的异步项目之一是为 Claude Code 实现 Vim 键位绑定。他们让 Claude 开发整个功能（尽管这并非优先事项），最终实现的内容中约有 70% 来自 Claude 的自主工作，仅需几次迭代就完成了开发。

测试生成与错误修复

他们在实现功能后使用 Claude Code 编写全面的测试，并处理拉取请求审查中发现的简单错误修复。他们还利用 GitHub Actions 集成，让 Claude 自动处理拉取请求评论，比如格式问题或函数重命名。

代码库探索

在处理不熟悉的代码库（比如单体仓库或 API 相关部分）时，团队会使用 Claude Code 快速了解系统的运作方式。他们不再等待 Slack 回复，而是直接向 Claude 询问解释和代码引用，大幅节省了上下文切换所耗费的时间。

用于产品开发的 Claude Code

团队影响

更快速的功能实现

Claude 自主编写了 70% 的代码，成功实现了 Vim 模式等复杂功能。

提升开发速度

能够快速制作功能原型并迭代想法，而不会陷入实现细节的泥潭。

通过自动化测试提升代码质量

Claude 生成全面的测试用例并处理常规的 bug 修复，在保持高标准的同时减少人工工作量。

更高效地探索代码库

团队成员无需等待同事回复，就能快速了解单代码仓库中不熟悉的部分。

Claude Code 团队的核心实践

打造自主循环

设置 Claude 以自动运行构建、测试和代码检查来验证其自身的工作。这能让 Claude 更自主地工作更久，并发现自身的错误，在你要求 Claude 先生成测试再编写代码时，这一方法尤为有效。

培养任务分类的直觉

学会区分适合异步处理的任务（外围功能、原型设计）与需要同步监督的任务（核心业务逻辑、关键修复）。产品边缘的抽象类任务可通过“自动接受模式”处理，而核心功能则需要更密切的监督。

制定清晰、详尽的提示词

当组件名称或功能相似时，你的请求务必足够具体。你的提示词越完善、越详细，就越能放心让 Claude 独立完成任务，而不会意外修改到代码库中错误的部分。

真正的安全领域的 Claude Code

真正的

安全工程团队专注于保障软件开发生命周期、供应链安全以及开发环境安全。他们广泛使用 Claude Code 来编写和调试代码。

Claude Code 主要应用场景

复杂基础设施调试

处理事件时，他们会向 Claude Code 提供堆栈跟踪和文档，让其追踪整个代码库的控制流。这显著缩短了生产环境问题的解决时间，使他们能在约 5 分钟内搞懂原本需要手动扫描代码 10-15 分钟才能发现的问题。

Terraform 代码审查与分析

对于需要安全批准的基础架构更改，他们将 Terraform 计划复制到 Claude Code 中，询问“这会做什么？我会后悔吗？”这会创建更紧密的反馈循环，并使安全团队更容易快速审查和批准基础架构更改，从而减少开发过程中的瓶颈。

文档生成和 Runbook

他们让 Claude Code 获取多个留档源，并创建降价运行手册、故障排除指南和概述。他们使用这些压缩文档作为调试实际问题的上下文，创建比搜索完整知识库更有效的工作流程。

测试驱动开发工作流程

他们现在向克劳德代码询问伪代码，指导它完成测试驱动开发，并在卡住时定期检查以指导它，从而产生更可靠和可测试的代码，而不是以前的“设计文档 - 简基代码 - 重构 - 放弃测试”模式。

上下文切换和项目入门

在为“dependant”（一款用于安全审批工作流的网络应用程序）等现有项目做贡献时，他们使用 Claude Code 来编写、审阅和执行以 Markdown 编写并存储在代码库中的规范，从而能在数天内而非数周内完成有价值的贡献。

用于安全领域的 Claude Code

真正的

团队影响

缩短事件解决时间

原本需要 10 到 15 分钟手动代码扫描的基础设施调试工作，现在大约只需要 5 分钟。

优化安全审核流程

用于安全审批的 Terraform 代码审核速度大幅提升，消除了开发人员等待安全团队审批的流程阻碍。

提升跨职能协作效率

团队成员无需花费数周时间建立背景，就能在数天内为项目做出有意义的贡献。

更完善的文档工作流程

从多个来源整合的故障排除指南和操作手册，可打造更高效的调试流程。

安全工程团队的核心技巧

广泛使用自定义斜杠命令

安全工程占整个单一代码库中所有自定义斜杠命令实现的 50%。这些自定义命令优化了特定工作流程，加快了重复任务的执行速度。

让 Claude 先进行外

他们不再针对代码片段提出针对性问题，而是现在让 Claude Code 做到“边做边提交你的工作”，并让它自主工作，只需定期进行检查，从而得出更全面的解决方案。

将其用于文档撰写

除了代码编写外，Claude Code 还擅长整合文档并生成结构化输出。用户可提供写作示例和格式偏好，从而获取可直接在 Slack、Google 文档及其他工具中使用的文档，避免因频繁切换界面而产生疲劳。

用于推理的 Claude 代码

推理团队负责管理内存系统，该系统会在 Claude 读取你的提示词并生成回复的过程中存储信息。团队成员，尤其是那些刚接触机器学习的人，可以大量使用 Claude Code 来弥补这一知识差距并加快工作进度。

Claude Code 主要应用场景

代码库理解与入职引导

该团队在接手复杂代码库时，高度依赖 Claude Code 来快速理解架构。他们不再手动搜索 GitHub 代码仓库，而是让 Claude 找出调用特定功能的文件，只需几秒就能得到结果，无需询问同事或手动搜索。

覆盖边缘情况的单元测试生成

写完核心功能后，他们让 Claude 编写全面的单元测试。Claude 会自动涵盖遗漏的边界情况，将通常需要耗费大量精力的工作在几分钟内完成，就像一位可供他们审核的编程助手。

机器学习概念讲解

没有机器学习背景的团队成員会依靠 Claude 来解释特定模型的功能和设置。原本需要花一小时通过谷歌搜索和阅读文档才能完成的事，现在只需要 10 到 20 分钟，研究时间减少了 80%。

陌生语言代码翻译

在测试不同编程语言的功能时，他们说明想要测试的内容，Claude 会用所需语言（如 Rust）编写逻辑，这样就无需为了测试而专门学习新的编程语言。

命令记忆与 Kubernetes 管理

他们不再需要记忆复杂的 Kubernetes 命令，而是向 Claude 询问正确的语法，比如“如何获取所有 Pod 或部署状态”，然后就能获得完成基础设施工作所需的具体命令。

用于推理的 Claude Code

团队影响

加速机器学习概念学习

研究时间缩短 80%——原本需要一小时的谷歌搜索，现在只需 10 到 20 分钟就能完成。

更快的代码库导航

能在几秒钟内找到相关文件并理解系统架构，而无需向同事请教。

全面的测试覆盖

Claude 会自动生成包含边界情况的单元测试，既减轻了脑力负担，又能保证代码质量。

消除语言障碍

可以在不学习 Rust 这类陌生编程语言的情况下实现相应功能。

推理团队的核心技巧

先测试知识库功能

试着提出各种问题，看看 Claude 是否能比谷歌搜索更快地给出答案。如果它的速度更快、结果更准确，那它就会成为你工作流程中一款宝贵的省时工具。

从代码生成开始

给 Claude 下达具体指令并让其编写逻辑，随后验证其正确性。这有助于在将该工具用于更复杂任务之前，建立对其能力的信任。

用它来编写测试

让 Claude 编写单元测试能大幅减轻日常开发工作的压力。利用这一功能，你无需花时间手动梳理所有测试用例，就能维持代码质量。

用于数据科学与可视化的 Claude

数据科学和机器学习工程团队需要复杂的可视化工具来了解模型性能，但构建这类工具通常需要掌握不熟悉的语言和框架专业知识。Claude Code 让这些团队无需成为全栈开发人员，就能打造生产级别的分析仪表盘。

Claude Code 主要应用场景

构建 JavaScript/TypeScript 仪表盘应用程序

尽管“对 JavaScript 和 TypeScript 知之甚少”，该团队仍使用 Claude Code 来构建用于可视化强化学习模型性能和训练数据的完整 React 应用程序。他们让 Claude 全权负责从零开始编写完整的应用程序，比如一个 5000 行的 TypeScript 应用程序，而自己无需理解代码本身。这一点至关重要，因为可视化应用程序的上下文相对有限，无需了解整个单体仓库，从而能够快速开发工具原型，以便在模型训练和评估过程中理解其性能表现。

处理重复性重构任务

当遇到合并冲突或半复杂的文件重构问题——这类问题对编辑器宏来说过于复杂，但又不值得投入大量开发精力时，他们会把 Claude Code 当作“老虎机”来使用——提交当前状态，让 Claude 自主工作 30 分钟，然后要么接受其解决方案，要么如果方案无效就重新开始。

打造可持续使用的分析工具，而非一次性的笔记本

团队不再编写一次性使用后就废弃的 Jupyter 笔记本，而是让 Claude 制作可在未来模型评估中重复使用的永久性 React 仪表盘。这一点至关重要，因为了解 Claude 的性能是“团队最重要的工作之一”——他们需要掌握模型在训练和评估过程中的表现，而这“其实并非易事，简单的工具仅通过观察单个数值的上升也无法获取太多有效信息”。

委托陌生任务

对于完全陌生的代码库或编程语言中的任务，他们将整个实现工作委托给 Claude Code，借助其能从单体代码仓库中收集上下文的能力，无需亲自参与实际编码流程即可执行任务。这让他们能在自身专业领域之外的领域保持高效，而非花费时间去学习新的技术。

用于数据科学与可视化的 Claude

团队影响

实现了 2-4 倍的时间节省

以往那些繁琐但手动也能完成的常规重构任务，现在完成速度大幅提升。

用陌生语言构建复杂应用程序

尽管只有极少的 JavaScript/TypeScript 经验，仍开发了 5000 行的 TypeScript 应用程序。

从一次性任务转向持久化任务

不再使用一次性的 Jupyter 笔记本，如今正构建可复用的 React 仪表板用于模型分析。

直接的模型优化建议

亲身参与 Claude Code 的开发经验，为后续模型版本的记忆系统优化和用户体验提升提供了参考。

实现了可视化驱动的政策制定

借助先进的数据可视化工具，能更好地了解 Claude 在训练与评估过程中的表现。

数据科学与机器学习工程团队的核心技巧

把它当成老虎机

在让 Claude 工作前保存你的状态，让它运行 30 分钟，然后要么接受结果要么重新开始，而不是费劲去修改。重新开始通常比试图修正 Claude 的错误成功率更高。

必要时为简化操作而中断

监督时，别犹豫停下 Claude 并问“你为什么要这么做？试试更简单的方法。”该模型默认倾向于更复杂的解决方案，但对简化方法的请求回应效果很好。

API 版 Claude Code

API 知识团队负责开发 PDF 支持、引用和网络搜索等功能，这些功能能将额外的知识纳入 Claude 的上下文窗口。在庞大复杂的代码库上开展工作，意味着经常会遇到不熟悉的代码段，需要花费大量时间明确针对特定任务需要查看哪些文件，并在进行修改前梳理清楚上下文。Claude Code 优化了这一工作体验，它能作为指导工具，帮助团队理解系统架构、定位相关文件，并解释复杂的交互逻辑。

Claude Code 的主要应用场景

第一步工作流规划

该团队将 Claude Code 作为处理任何任务的“首选工具”，让它确定哪些文件需要检查，以进行错误修复、功能开发或分析。这取代了传统的耗时流程——即在开始工作前手动浏览代码库并收集相关背景信息。

跨代码库独立调试

团队现在有信心去处理代码库中陌生部分的漏洞，而不是向他人求助。他们可以向 Claude 询问“你觉得能修复这个漏洞吗？这是我遇到的情况”，并且往往能立刻取得进展，而在过去，这需要投入大量时间，是不太现实的。

通过内部试田进行模型迭代测试

Claude Code 会自动使用最新的研究模型快照，这使其成为用户体验模型更新的主要方式。这让用户能在开发周期中直接获得模型行为变化的反馈，而这是他们在之前的版本发布中从未体验过的。

消除上下文切换的开销

无需再复制代码片段、向 Claude.ai 拖拽文件并详细解释问题，用户可直接在 Claude Code 中提问，无需额外收集上下文，大幅降低精神负担。

Claude Code 接口

团队影响

提升应对陌生领域的信心

团队成员能够独立调试陌生代码库中的漏洞并调查相关事件。

大幅提升上下文构建时间

消除了复制代码片段和将文件拖入 Claude.ai 的操作，减轻了思维上下文切换的负担。

更快的轮岗入门

工程师轮岗到新团队后，无需大量同事协助就能快速熟悉陌生的代码库，并做出有价值的贡献。

提升开发者幸福感

团队成员反馈，日常工作流程中的阻碍减少后，他们的幸福感和工作效率都有所提升。

API 知识团队的核心建议

将其视为迭代式的合作伙伴，而非一次性的解决方案

与其期待 Claude 立刻解决问题，不如把它当作可以一起迭代的合作者。这种方式比试图一次就得到完美解决方案效果更好。

用它来建立在陌生领域的信心

别犹豫，去解决超出你专业领域的漏洞或调查事件吧——Claude Code 让你能够在那些通常需要大量上下文构建的领域独立开展工作。

从小信自开始

从你仅需的最基础内容开始，让 Claude 引导你完成整个过程，而不是提前铺陈大量的解释说明。

用于增长营销的 Claude

Code

增长营销团队专注于搭建付费搜索、付费社交、移动应用商店、电子邮件营销和搜索引擎优化等效果营销渠道。作为一个非技术型的单人团队，他们借助 Claude Code 自动化重复性的营销任务，并构建智能工作流，而这类工作传统上需要大量的工程资源支持。

Claude Code 主要应用场景

谷歌广告创意自动化生成

该团队构建了一套智能体工作流，可处理包含数百条现有广告及相关效果指标的 CSV 文件，识别出表现不佳的广告以进行优化迭代，并生成符合严格字符限制的全新变体（标题 30 个字符、描述 90 个字符）。系统配备两个专门的子智能体（一个负责标题，一个负责描述），能在数分钟内生成数百条新广告，无需人工为多个广告活动逐一创建。这让他们得以大规模开展测试与迭代，而这在过去需要耗费大量时间才能实现。

用工世界创意制作的 Figma 插件

他们不再为付费社交广告手动复制编辑静态图片，而是开发了一款 Figma 插件，该插件可识别画面帧，并通过替换标题和描述，以编程方式生成多达 100 个广告变体。这将原本需要数小时的复制粘贴工作，缩短为每批次仅需半秒。这实现了十倍的创意产出，让团队能够在核心社交渠道上测试更多样化的创意变体。

用工广告活动分析的 Meta Ads MCP 服务器

他们搭建了一个集成 Meta Ads API 的 MCP 服务器，可直接在 Claude 桌面应用中查询广告系列表现、支出数据以及广告效果，无需在多个平台间切换进行效果分析，节省了宝贵时间——每一次效率提升都能转化为更高的投资回报率。

结合记忆系统的高级提示词工程

他们搭建了一套基础记忆系统，可记录广告迭代过程中的假设与实验数据，让系统在生成新变体时能将过往测试结果纳入上下文参考，由此构建出一个可自我优化的测试框架。这使得系统化的实验成为可能，而这类实验若靠人工追踪则根本无法实现。

用于增长营销的 Claude Code

团队影响力

大幅节省重复任务的时间

广告文案创作时间从 2 小时缩短至 15 分钟，为策略类工作腾出了时间。

创意产出提升 10 倍

借助自动化生成功能和 Figma 集成，团队现在可以在各个渠道上测试数量大幅增加的广告变体了。

运营团队堪比更大的团队

团队如今能够处理那些传统上需要专门工程资源来完成的任务。

战略重心转移

团队可以将更多时间投入到整体战略制定和智能体自动化构建上，而非手动执行。

增长营销团队的核心技巧

识别适合 API 的重复性任务

寻找涉及使用带有 API 的工具（如广告平台、设计工具、分析平台）进行重复操作的工作流程。这些是自动化的理想对象，也是 Claude Code 能发挥最大价值的场景。

将复杂的工作流程拆分为专门的子智能体

与其试图在一个提示词或工作流中处理所有任务，不如为特定任务创建独立的智能体（比如标题生成智能体和描述生成智能体）。这样能让调试更轻松，在处理复杂需求时还能提升输出质量。

编码前进行充分的头脑风暴并规划提示词

前期花大量时间通过 Claude.ai 梳理整个工作流程，然后让 Claude.ai 生成一份全面的提示词和代码结构，供 Claude Code 参考。同时，要按步骤推进工作，而非一次性索要完整解决方案，避免 Claude 因复杂任务而不堪重负。

用于产品设计的 Claude Code

产品设计团队为 Claude Code、Claude.ai 以及 Anthropic API 提供支持，专注于打造人工智能产品。即便是非开发人员也能使用 Claude Code，打破设计与工程之间的传统壁垒，无需与工程师进行大量反复沟通，即可直接实现自己的设计愿景。

Claude Code 主要应用场景

前端优化与状态管理调整

他们不再需要制作详尽的设计文档，也不用为视觉调整（字体、颜色、间距）和工程师进行多轮反馈，如今直接通过 Claude Code 就能实现这些修改。工程师们表示，他们正在进行“大型状态管理修改，这类修改通常不会由设计师来做”，这让他们能够达成自己设想的精准效果。

GitHub Actions 自动化工单

借助 Claude Code 的 GitHub 集成功能，开发者只需提交描述所需修改的问题 / 工单，Claude 就能自动提出代码解决方案，无需打开 Claude Code，为积压的优化任务打造了无缝的漏洞修复与功能优化工作流程。

快速原型生成与设计

将原型图粘贴到 Claude Code 中，它会生成功能完整的原型，工程师可以立即理解并对其进行迭代，取代了传统的静态 Figma 设计流程——传统流程中需要大量解释并将设计转化为可运行的代码。

快速实现原型与系统架构理解

他们借助 Claude Code 梳理错误状态、逻辑流程以及各类系统状态，得以在设计阶段就识别边缘情况，而非等到开发后期才发现，从根本上提升了初始设计的质量。

复杂文案修改与合规审查

对于在整个代码库中移除“研究预览”提示信息这类任务，他们使用 Claude Code 查找所有相关实例、审核周边文案、与法务实时协调变更并执行更新——这一过程仅需两次 30 分钟的通话，而非耗时一周的反复沟通。

用于产品设计的 Claude Code

团队影响

核心工作流程实现革新

Claude Code 成为主要设计工具，Figma 和 Claude Code 的打开时长占比达 80%。

执行效率提升 2-3 倍

以前需要与工程师反复沟通才能完成的视觉和状态管理修改，现在可直接实现。

11 分钟完成原本需 1 小时的任务

像 GA 这样的复杂项目，原本需要一周协调才能完成的消息发布，现在通过两场 30 分钟的会议就搞定了。

工程师赋能了不同的用户群体

开发人员获得“增强型工作流”（执行速度更快），而非技术用户则体验到“哇，我居然用上了开发人员的工作流”（拥有了此前完全不可能实现的全新功能）。

优化设计与工程协作

沟通更顺畅，问题解决效率更高，因为设计师能提前了解系统的限制与可能性。

产品设计团队的核心技巧

从工程师那里获得完美的部署协助

让工程团队协助完成代码仓库的初始设置和权限配置——非开发人员在技术入职环节会面临挑战，但一旦配置完成，日常工作流程将迎来质的提升。

通过自定义记忆文件引导 Claude 的行为

制定具体指令，向 Claude 说明你是一名几乎没有编程经验的设计师，需要详细的解释以及更细微的逐步调整，这将显著提升 Claude 回复的质量，也会让它的回答更易理解。

利用设计图快速生成原型代码

使用 Command+V 快捷键可将截图直接粘贴到 Claude Code 中——它擅长解读设计图并生成功能性代码，能将静态原型快速转化为交互式原型，工程师可直接理解并在此基础上开发。

用于强化学习工程的 Claude Code

强化学习工程团队专注于强化学习中的高效采样以及集群间的权重迁移。他们主要使用 Claude Code 来编写中小型功能、调试代码和理解复杂代码库，采用的迭代方式包含频繁的检查点保存与回滚操作。

Claude Code 主要应用场景

监督式自主开发的功能构建

该团队让 Claude Code 负责编写中小型功能的大部分代码，同时提供监督，比如为权重转移组件实现身份验证机制。他们以交互方式开展工作，让 Claude 主导开发，但在其偏离方向时进行引导。

测试生成与代码审查

他们自行完成修改后，会让 Claude Code 编写测试或审查代码。这种自动化测试 workflow 为日常但重要的质量保证任务节省了大量时间。

调试与错误排查

他们使用 Claude Code 调试错误，效果有好有坏——有时它能立即识别问题并添加相关测试，而有时则难以理解问题，但整体而言，它能正常发挥作用时还是能提供价值。

代码库理解与调用栈分析

他们工作流程中最大的变化之一是使用 Claude Code 快速获取相关组件和调用堆栈的摘要，以此替代手动阅读代码或生成大量调试输出。

Kubernetes 运维也已

他们经常向 Claude Code 咨询原本需要大量搜索才能解决的 Kubernetes 操作问题，能立刻得到配置和部署相关问题的答案。

用于强化学习工程的 Claude Code

开发工作流程影响

实验方法得以实现

他们现在采用了一种“尝试与回滚”的方法，频繁提交检查点，以便测试 Claude 的自主实现尝试，并在必要时进行回滚，从而支持开展更多实验性工作。

开发效率提升

Claude Code 会自动添加实用注释，能大幅节省编写文档的时间，不过用户反馈它有时会在奇怪的位置添加注释，或采用不太合理的代码组织方式。

提速但存在局限

尽管 Claude Code 能在人类投入“相对较少时间”的情况下完成中小型代码合并请求（PR）的开发，但他们也承认，该工具一次就能成功完成任务的概率仅约三分之一，要么需要额外的指导，要么得进行人工干预。

强化学习工程团队的核心技巧

针对特定模式编写你的 Claude.md 文件

在你的 Claude.md 文件中添加指令，以防止 Claude 反复出现工具调用错误，比如告诉它“运行 pytest 时不要执行多余操作，也不要不必要地切换目录——直接使用正确的路径”。这一做法显著提升了操作的一致性。

应用实验检查点工作流

在 Claude 进行修改时定期提交你的工作，这样当实验失败时你可以轻松回滚。这让开发能采用更具实验性的方法且无风险。

先尝试一次性完成，再协作推进

给 Claude 一个简短的提示，先让它尝试完成完整的实现。如果成功了（大约有一分之一的概率），你就能节省大量时间。如果没成功，再切换到更协作式、有引导的方法。

Claude Code 法律应用

法务团队通过试验以及想要了解 Anthropic 产品体系的想法，发现了 Claude Code 的潜力。此外，有一名团队成员有一个个人使用场景，即开发面向家人的辅助工具，以及打造能向非开发者展示该技术强大功能的工作原型。

Claude Code 的主要应用场景

为家人定制的可无障碍解决方案

团队成员为那些因医学诊断而存在语言障碍的家属开发了沟通辅助工具。他们仅用一小时就利用原生语音转文字功能打造了一款预测文本应用，该应用能给出回复建议并通过语音库进行语音播报，弥补了言语治疗师推荐的现有辅助工具存在的不足。

法务部门工作流程自动化

他们打造了原型“电话树”系统，帮助团队成员联系到 Anthropic 公司合适的律师，这展示了法务部门如何无需传统开发资源，就能为常见工作搭建定制工具。

团队协调工具

管理人员开发了 G Suite 应用程序，可自动完成每周团队更新并跟踪各产品的法律审核状态，这让律师只需简单点击按钮就能快速标记需要审核的事项，无需再通过电子表格进行管理。

用工安全验证的合规原型开发

他们使用 Claude Code 快速构建可用的原型，展示给领域专家（比如向加州大学旧金山分校的专家展示辅助工具），以验证想法并在投入更多时间前找到现有的解决方案。

法律领域的 Claude Code

工作风格与影响力

在 Claude.ai 中规划 在 Claude Code 中构建

他们采用两步流程：首先与 Claude.ai 一起进行头脑风暴和规划，然后转向 Claude Code 进行实施，要求其放慢速度、分步推进，而非一次性输出所有内容。

视觉化生成

他们经常使用截图向 Claude Code 展示自己想要的界面样式，然后根据视觉反馈进行迭代，而不是用文字描述功能。

原型驱动的创新

他们强调要克服分享“愚蠢”或“玩具级”原型的恐惧，因为这些演示能激励他人发现自己未曾考虑过的可能性。

安全与合规意识

MCP 集成相关顾虑

作为产品律师，他们立即识别出深度集成多企业通信平台（MCP）所带来的安全影响，并指出随着人工智能工具访问更敏感的系统，保守的安全策略将形成阻碍。

合规工具优先事项

随着人工智能能力不断拓展，他们主张快速构建合规工具，同时兼顾创新与风险管理之间的平衡。

法务部门的核心建议

先在 Claude.ai 中进行充分规划

使用 Claude 的对话界面完善你的整个想法，再转向 Claude Code。然后让 Claude 将所有内容总结为一份可执行的分步提示词。

循序渐进且注重视觉呈现

让 Claude Code 放慢速度，一步一步执行，这样你就能复制粘贴而不会感到不知所措。大量使用截图来展示你希望界面呈现的样子。

避免完美主义 公开原型

克服想要隐藏“玩具级”项目或未完成工作的冲动——分享原型能帮助他人看到各种可能性，并在平时缺乏互动的部门之间激发创新。

AI